

HUNDRED DAYS OF CODE

DAY 10

QUESTION 1: WRITE A C PROGRAM TO INPUT TIME IN SECONDS AND CONVERT IT INTO DAYS, HOURS, MINUTES, AND SECONDS FORMAT.

STEP 1: Required input, output, and formula.

INPUT: A time value in seconds.

Output: the equivalent time in days, hours, minutes, seconds.

FORMULA: We know: 1 day = $24 \times 60 \times 60 = 86400$ seconds

1 hour = $60 \times 60 = 3600$ seconds

1 minute = 60 seconds

STEP 2: ALGORITHM:

1. START
2. Declare variables – totalseconds, days, hours, minutes, and seconds.
3. Input the total number of seconds.
4. Calculate the number of days using: $\text{days} = \text{total seconds} / 86400$.
5. Calculate the number of hours using: remainder from above division / 3600.
6. Calculate the number of minutes using: remainder from above division / 60.
7. Calculate the number of seconds using: remainder from above division / 60.
8. Display days, hours, minutes, and seconds.
9. STOP

STEP 3: PSEUDOCODE:

START

Input totalseconds

Days = $\text{totalseconds} / 86400$

remaining = $\text{totalseconds} \% 86400$

Hours = $\text{remaining} / 3600$

remaining = $\text{remaining} \% 3600$

Minutes = $\text{remaining} / 60$

Seconds = $\text{remaining} \% 60$

DISPLAY days

DISPLAY HOURS

DISPLAY MINUTES

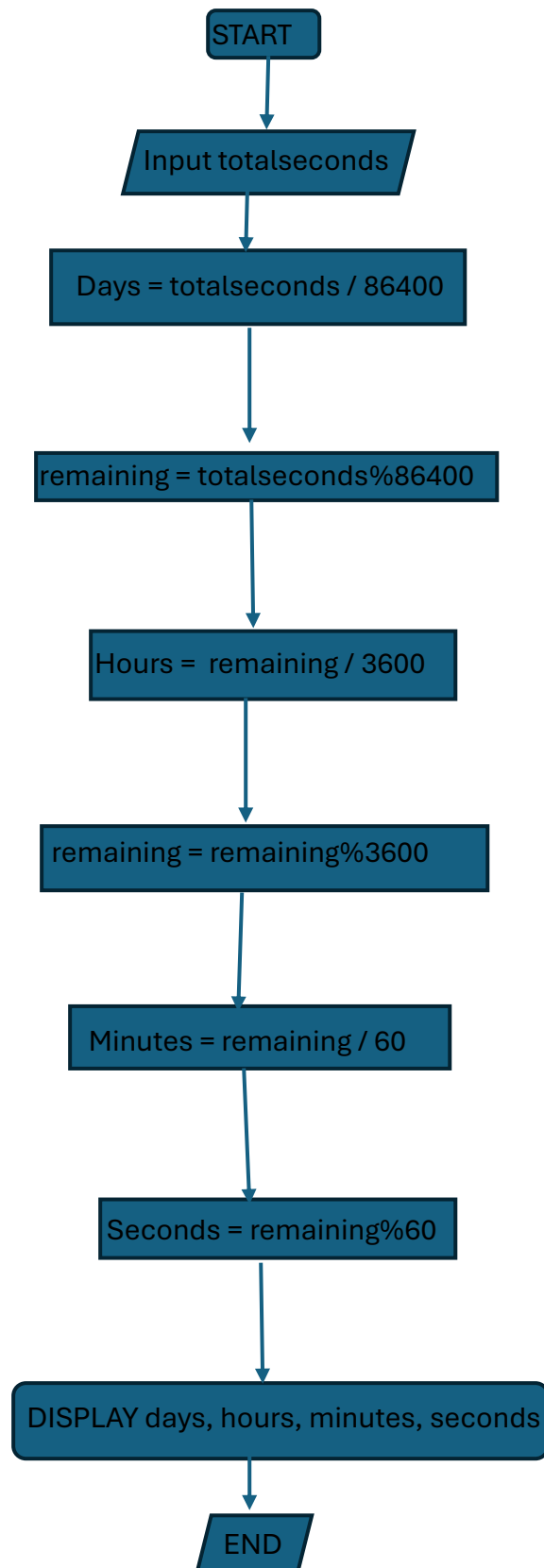
HUNDRED DAYS OF CODE

DAY 10

DISPLAY seconds

END

FLOWCHART:



HUNDRED DAYS OF CODE

DAY 10

STEP 4: REQUIRED C PROGRAM

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int totalSeconds;
```

```
    int days, hours, minutes, seconds;
```

```
    int remaining;
```

```
    printf("Enter time in seconds: ");
```

```
    scanf("%d", &totalSeconds);
```

```
    days = totalSeconds / 86400;
```

```
    remaining = totalSeconds % 86400;
```

```
    hours = remaining / 3600;
```

```
    remaining = remaining % 3600;
```

```
    minutes = remaining / 60;
```

```
    seconds = remaining % 60;
```

```
    printf("\nTime = %d days, %d hours, %d minutes, %d seconds\n",
```

```
        days, hours, minutes, seconds);
```

```
    return 0;
```

```
}
```

HUNDRED DAYS OF CODE

DAY 10

STEP 5: TEST CASE TABLE

TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	60	0 days, 0 hours, 1 minute, 0 seconds	0 days, 0 hours, 1 minute, 0 seconds	PASS
2	3600	0 days, 1 hours, 0 minute, 0 seconds	0 days, 1 hours, 0 minute, 0 seconds	PASS
3	90061	1 day, 1 hour, 1 minute, 1 second	1 day, 1 hour, 1 minute, 1 second	PASS

STEP 6: COMPILATION AND EXECUTION

```
(kali㉿kali)-[~]  
$ cc time.c -o time  
  
(kali㉿kali)-[~]  
$ ./time  
Enter time in seconds: 60  
  
Time = 0 days, 0 hours, 1 minutes, 0 seconds  
  
(kali㉿kali)-[~]  
$ ./time  
Enter time in seconds: 3600  
  
Time = 0 days, 1 hours, 0 minutes, 0 seconds  
  
(kali㉿kali)-[~]  
$ ./time  
Enter time in seconds: 90061  
  
Time = 1 days, 1 hours, 1 minutes, 1 seconds  
  
(kali㉿kali)-[~]  
$
```

HUNDRED DAYS OF CODE

DAY 10

QUESTION 2: Write a program to input a year and check whether it is a leap year or not.
GIVE ME THE STEP BY STEP CONTENTS AND PROCESS TO COMPLETE THE ASSIGNMENT

STEP 1: INPUT, OUTPUT, FORMULA

Input:

A year.

Output:

Whether the year is a leap year or not.

Leap-year rule

A year is a leap year if:

- It is divisible by 400,

OR

- It is divisible by 4 **and not divisible by 100.**

STEP 2: ALGORITHM

☐ Start.

☐ Declare an integer variable year.

☐ Input the year.

☐ Check whether $\text{year \% } 400 == 0$.

☐ If true, display "Leap Year".

☐ Otherwise, check whether $\text{year \% } 4 == 0$ and $\text{year \% } 100 != 0$.

☐ If true, display "Leap Year".

☐ Otherwise, display "Not a Leap Year".

☐ Stop.

STEP 3: PSEUDOCODE:

INPUT year

IF $\text{year \% } 400 == 0$ THEN

 DISPLAY "Leap Year"

HUNDRED DAYS OF CODE

DAY 10

```
ELSE IF year % 4 == 0 AND year % 100 != 0 THEN
```

```
    DISPLAY "Leap Year"
```

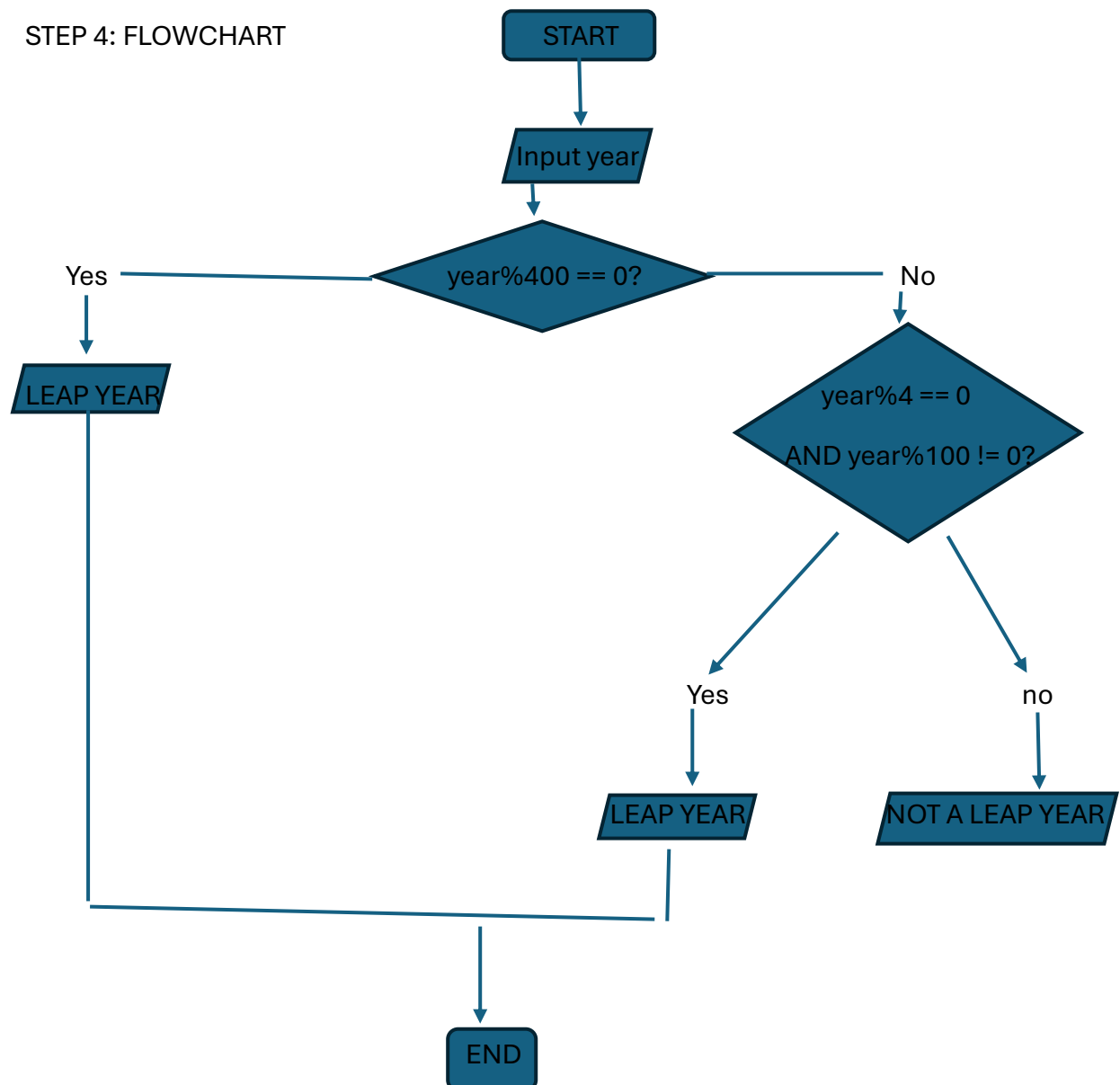
```
ELSE
```

```
    DISPLAY "Not a Leap Year"
```

```
END IF
```

```
END
```

STEP 4: FLOWCHART



HUNDRED DAYS OF CODE

DAY 10

STEP 5: REQUIRED C PROGRAM

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int year;
```

```
    printf("Enter a year: ");
```

```
    scanf("%d", &year);
```

```
    if (year % 400 == 0)
```

```
    {
```

```
        printf("%d is a leap year.\n", year);
```

```
    }
```

```
    else if (year % 4 == 0 && year % 100 != 0)
```

```
    {
```

```
        printf("%d is a leap year.\n", year);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("%d is not a leap year.\n", year);
```

```
    }
```

```
    return 0;
```

```
}
```

HUNDRED DAYS OF CODE

DAY 10

STEP 6: TEST CASE TABLE:

TEST CASE	INPUT YEAR	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	2024	LEAP YEAR	LEAP YEAR	PASS
2	1900	NOT A LEAP YEAR	NOT A LEAP YEAR	PASS
3	2000	LEAP YEAR	LEAP YEAR	PASS

STEP 7: COMPIILATION AND EXECUTION

```
(kali㉿kali)-[~]
└─$ vi year.c

(kali㉿kali)-[~]
└─$ cc year.c -o year

(kali㉿kali)-[~]
└─$ ./year
Enter a year: 1900
1900 is not a leap year.

(kali㉿kali)-[~]
└─$ 2024
2024: command not found

(kali㉿kali)-[~]
└─$ ./year
Enter a year: 2024
2024 is a leap year.

(kali㉿kali)-[~]
└─$ ./year
Enter a year: 2000
2000 is a leap year.
```

.....